Run a Simulation before placing your next Polymarket bet - Code included.

The Monte Carlo method is hyped across Polymarket Quants right now.

But only a few knows how to apply it correctly to different situations on PM to run a simulation based on it.

Run these 4 simulations before placing your next bet:

> Sequential Monte Carlo simulation

Used for dynamic simulation when a market has several parameters, and each of them can affect the result

• Logit random walk formula:

$\text{logit}(x_t) = \text{logit}(x_{t-1}) + \varepsilon_t$, where $\varepsilon_t \sim N(0, \sigma^2_process)$

//

> Tail events simulation

Tail events - events with very low probability, and they're hard to simulate reliably.

However, you can't ignore them.

• Formula for estimating a tail event probability:

$\hat{p}_IS = (1/N) \Sigma_\{i=1..N\} 1\{X_i \in A\} \cdot w(X_i)$, with $X_i \sim Q$

//

> Correlated markets outcomes simulation:

Tail dependence: markets co-move much more in extremes than correlation implies.

• Upper tail dependence formula:
$\lambda_U = \text{limit as } u \to 1^- \text{ of } P(Y > FY^{-1}(u) \mid X > FX^{-1}(u))$

• Lower tail dependence formula:
$\lambda_L = \text{limit as } u \to 0^+ \text{ of } P(Y \le FY^{-1}(u) \mid X \le FX^{-1}(u))$

//

> Agent-Based simulation

Prediction markets have different agents: informed traders, noise traders, market makers, and bots.

Despite irrational traders, prices still tend to converge to fair value over time.

```
PredictionMarketParticleFilter:
"""
equential Monte Carlo filter for real-time event probability

sage during a live event (e.g., election night):
    pf = PredictionMarketParticleFilter(prior_prob=0.50)
    pf.update(observed_price=0.55)   # market moves on early
    pf.update(observed_price=0.62)   # more data
    pf.update(observed_price=0.58)   # partial correction
    print(pf.estimate())             # filtered probability
"""
ef __init__(self, N_particles=5000, prior_prob=0.5,
            process_vol=0.05, obs_noise=0.03):
    self.N = N_particles
    self.process_vol = process_vol
    self.obs_noise = obs_noise

    # Initialize particles around prior
    logit_prior = logit(prior_prob)
    self.logit_particles = logit_prior + np.random.normal(0,
    self.weights = np.ones(N_particles) / N_particles
    self.history = []

ef update(self, observed_price):
    """Incorporate a new observation (market price, poll resu
    # 1. Propagate: random walk in logit space
    noise = np.random.normal(0, self.process_vol, self.N)
    self.logit_particles += noise
```